



Heartbeat

От пинга до инструмента
управления системой

Александр Илларионов
.NET разработчик, Altenar
27.05.2026





Александр Илларионов

.NET разработчик

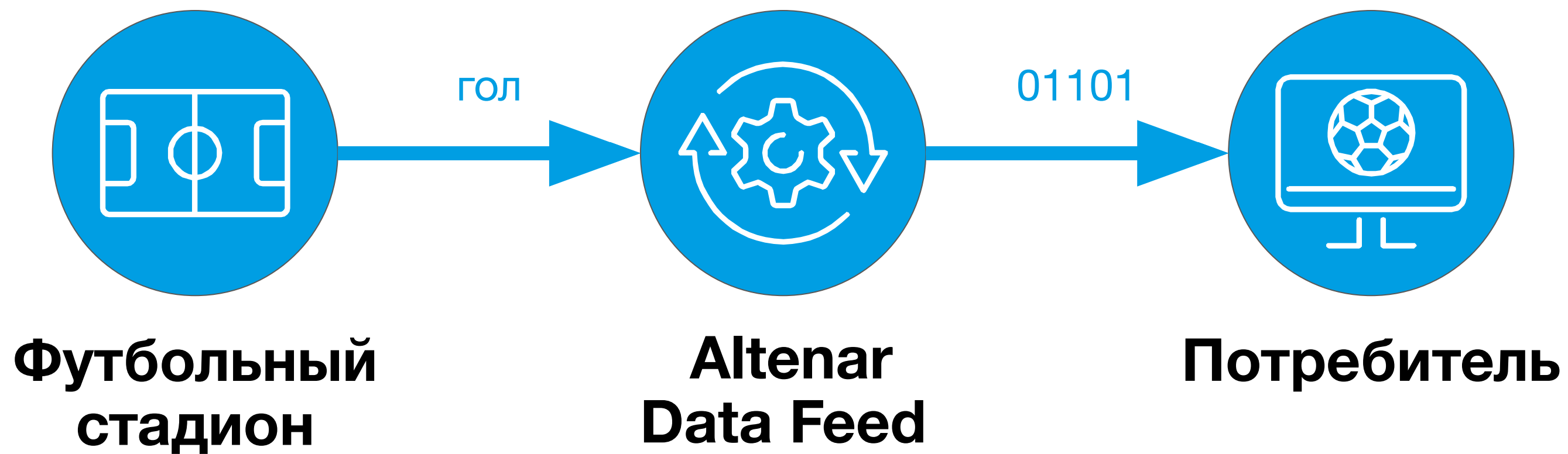


4+
Backend
программист

Работаю в команде,
отвечающей за подсистему
экспорта, включая heartbeat и
recovery



Наш проект



В начале было...

- Вы работник компании
- Собрались в субботу на шашлыки

В начале было...

- Вы работник компании
- Собрались в субботу на шашлыки
- Раздаются звонки и уведомления
- У ВАС ЧТО-ТО НЕ РАБОТАЕТ



В начале было...



Поздравляем! Вы создали Heartbeat



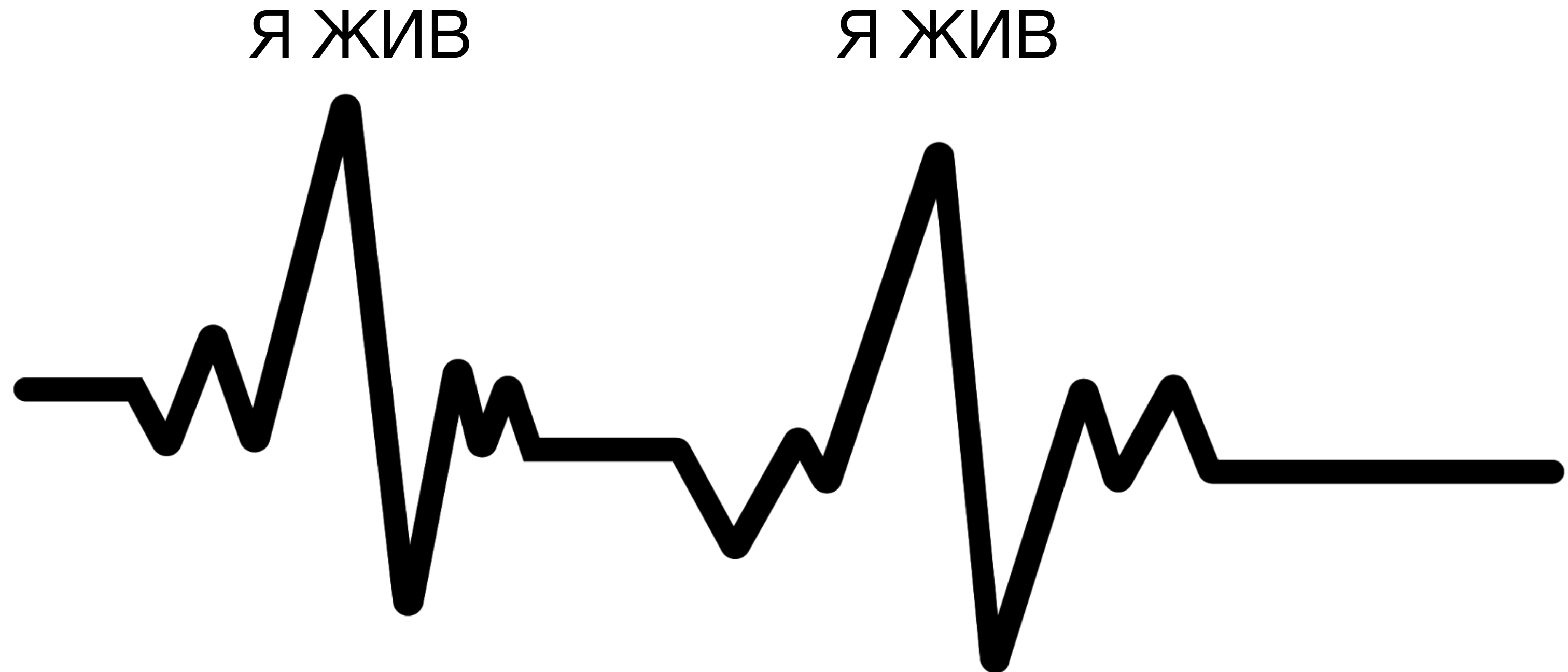
Так появился он... heartbeat в системе

- В каком-то смысле вы научились информировать пользователей
- Но почему именно heartbeat?

Heartbeat как сигнал о здоровье

ЭТОТ СЛАЙД НЕ ПУСТОЙ

Heartbeat как сигнал о здоровье



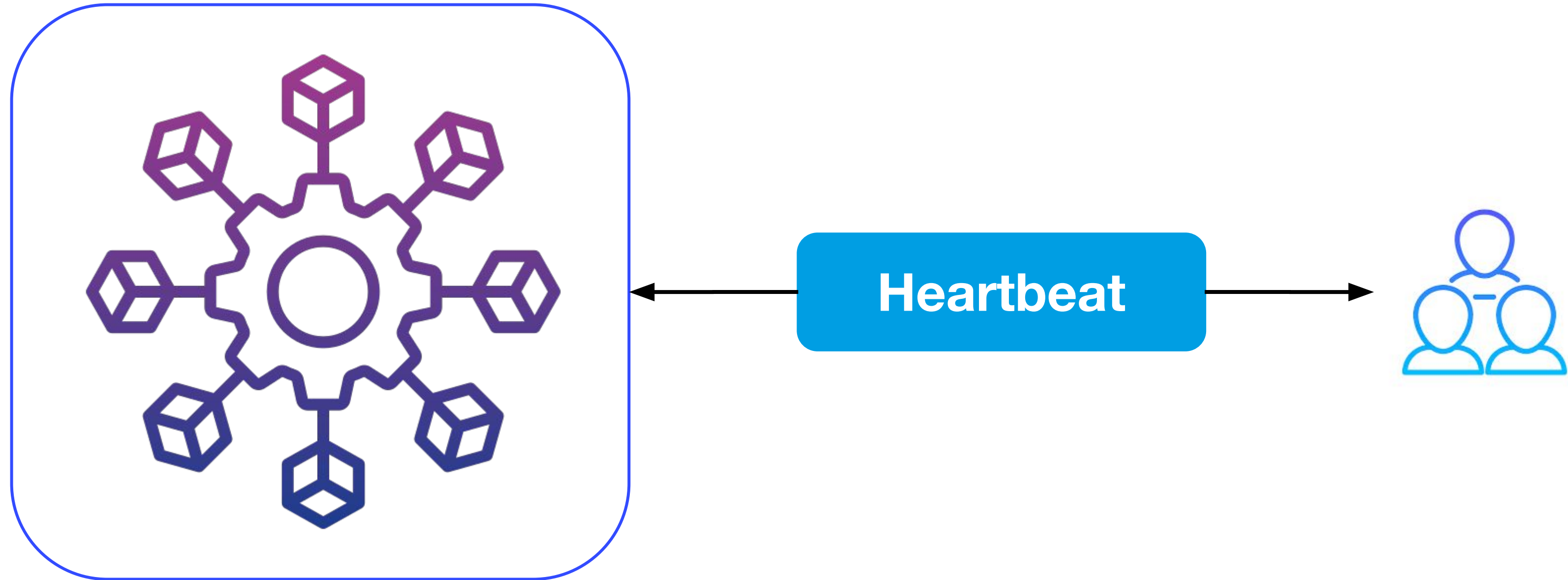
Разные уровни Heartbeat

Под **Heartbeat'ом** можно понимать разное в зависимости от контекста

- ICMP / Ping (На уровне сети)
- Keep-Alive (Для протоколов)
- Probes (Kubernetes)



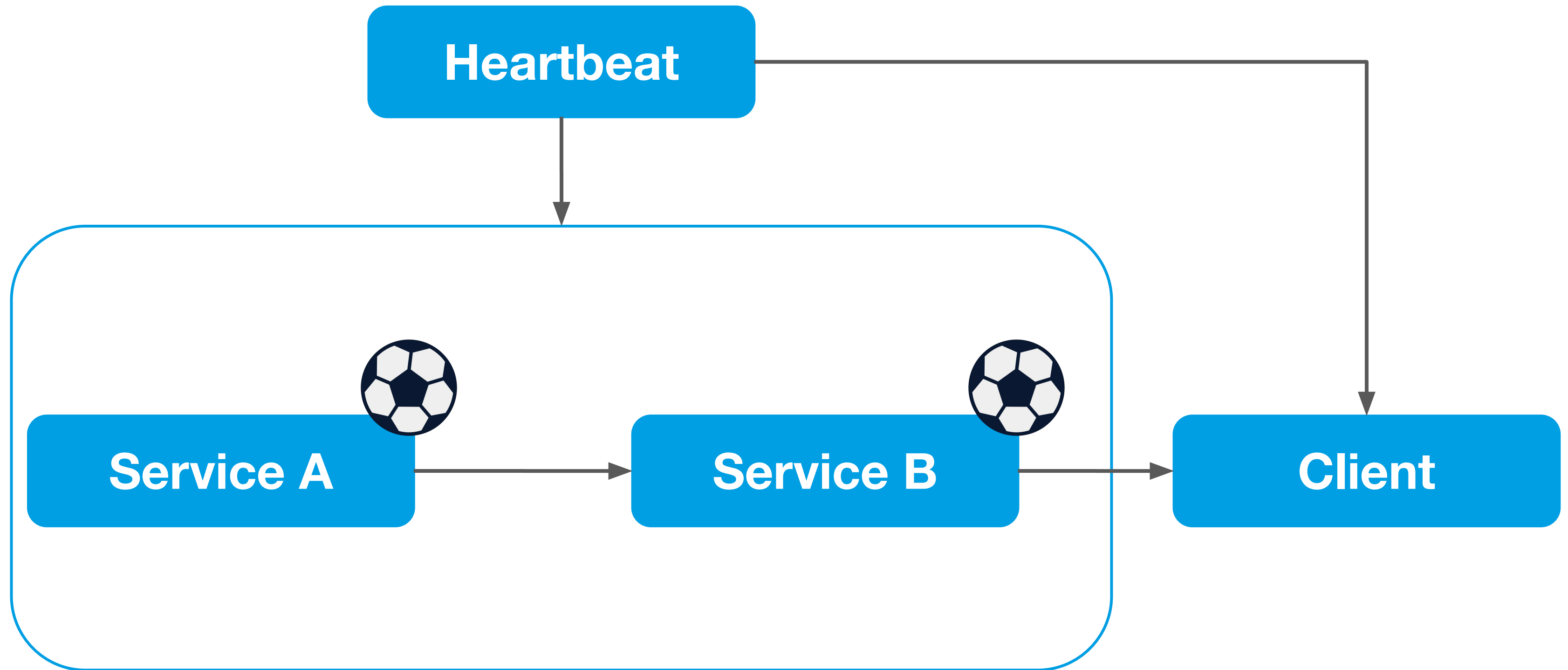
Что мы понимаем под Heartbeat



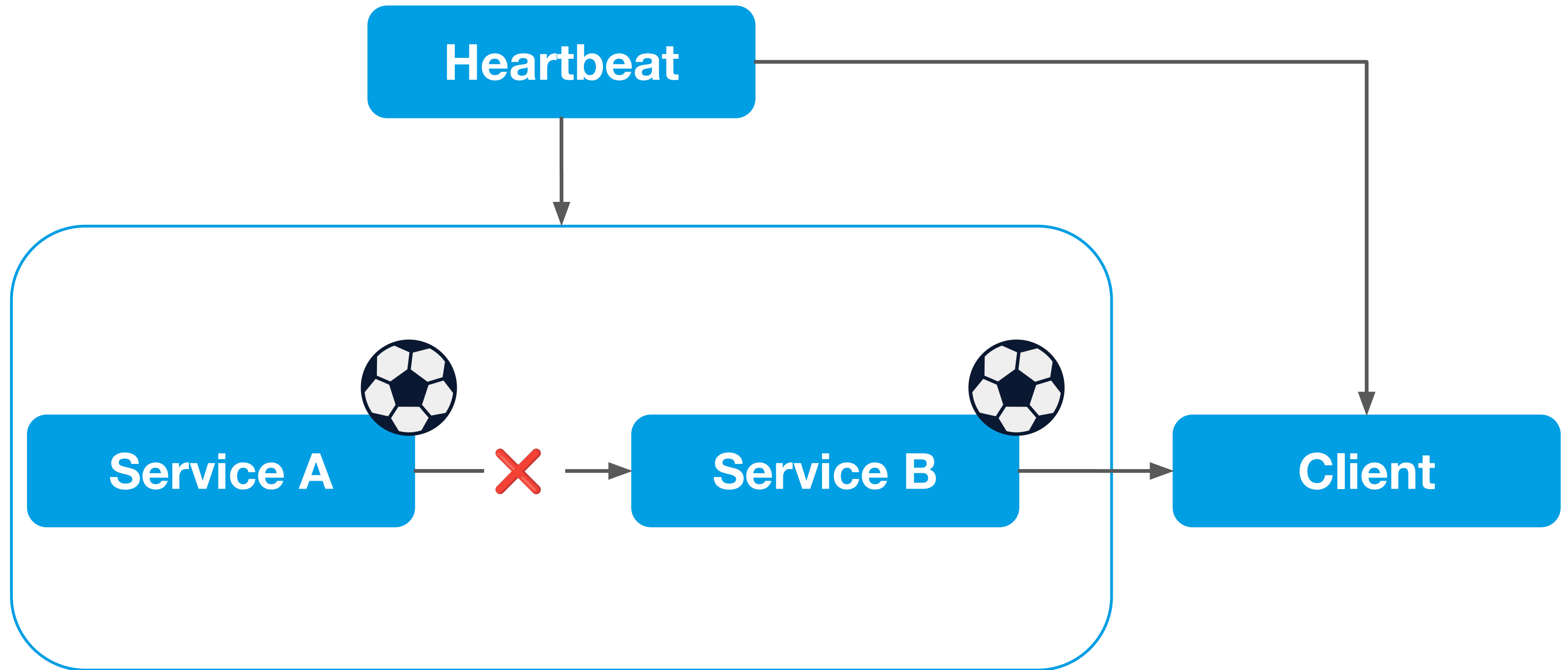
Heartbeat как сигнал о здоровье

Heartbeat – это периодический сигнал, посылаемый для подтверждения работоспособности.

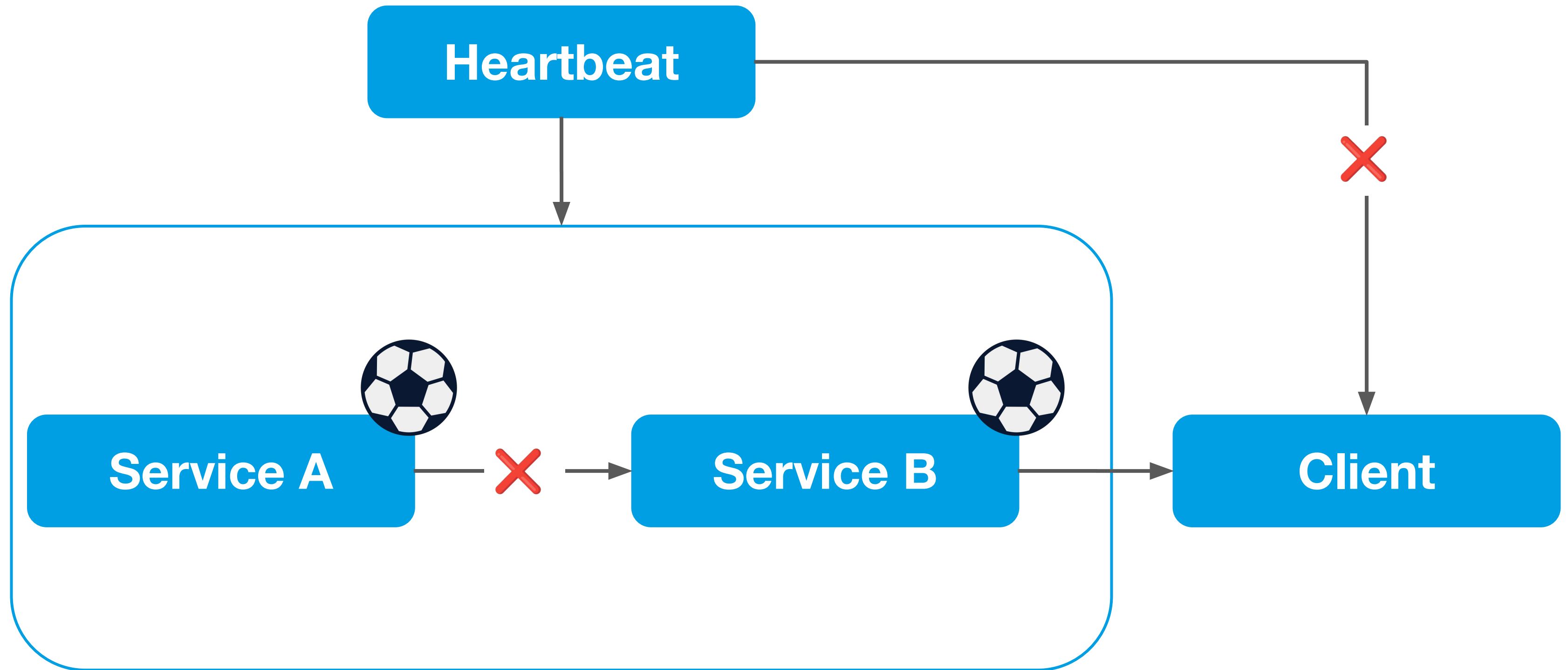
Как было у нас



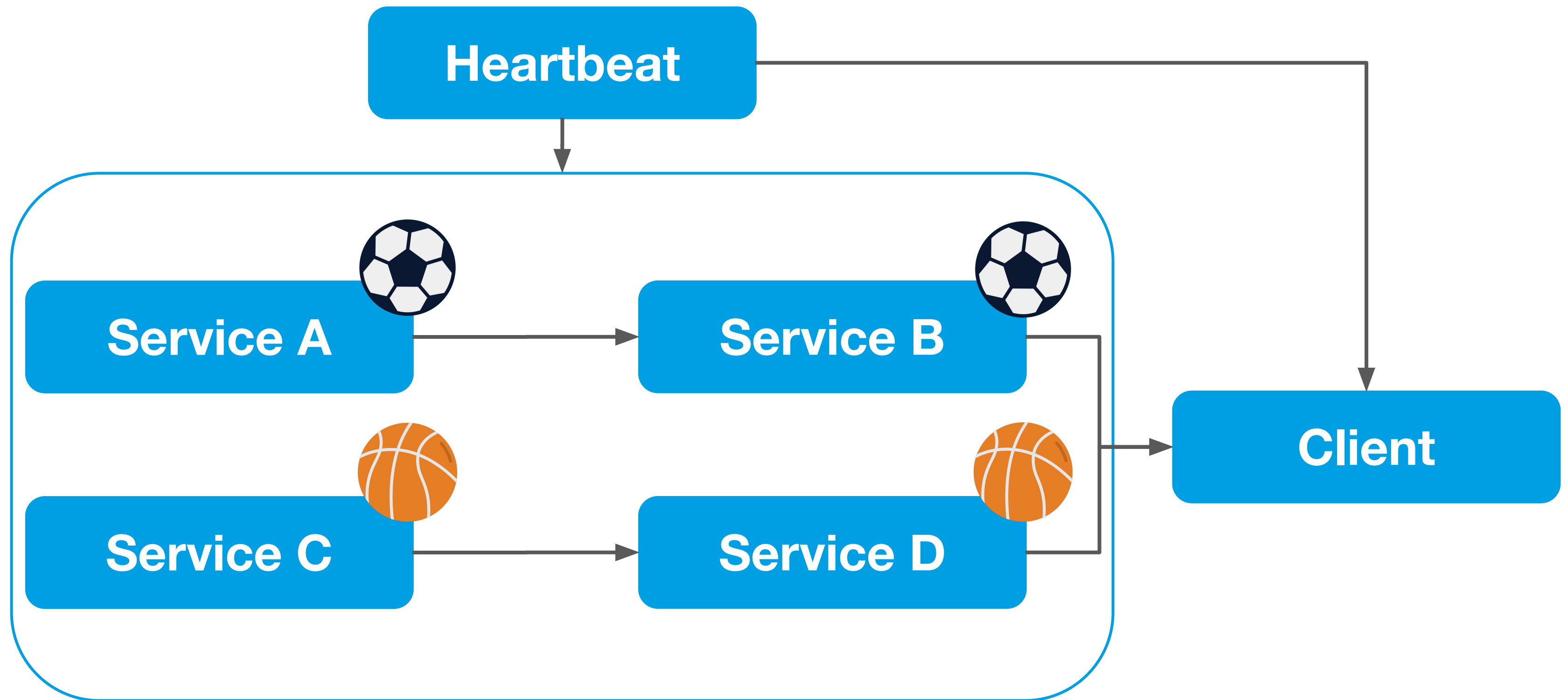
Как было у нас



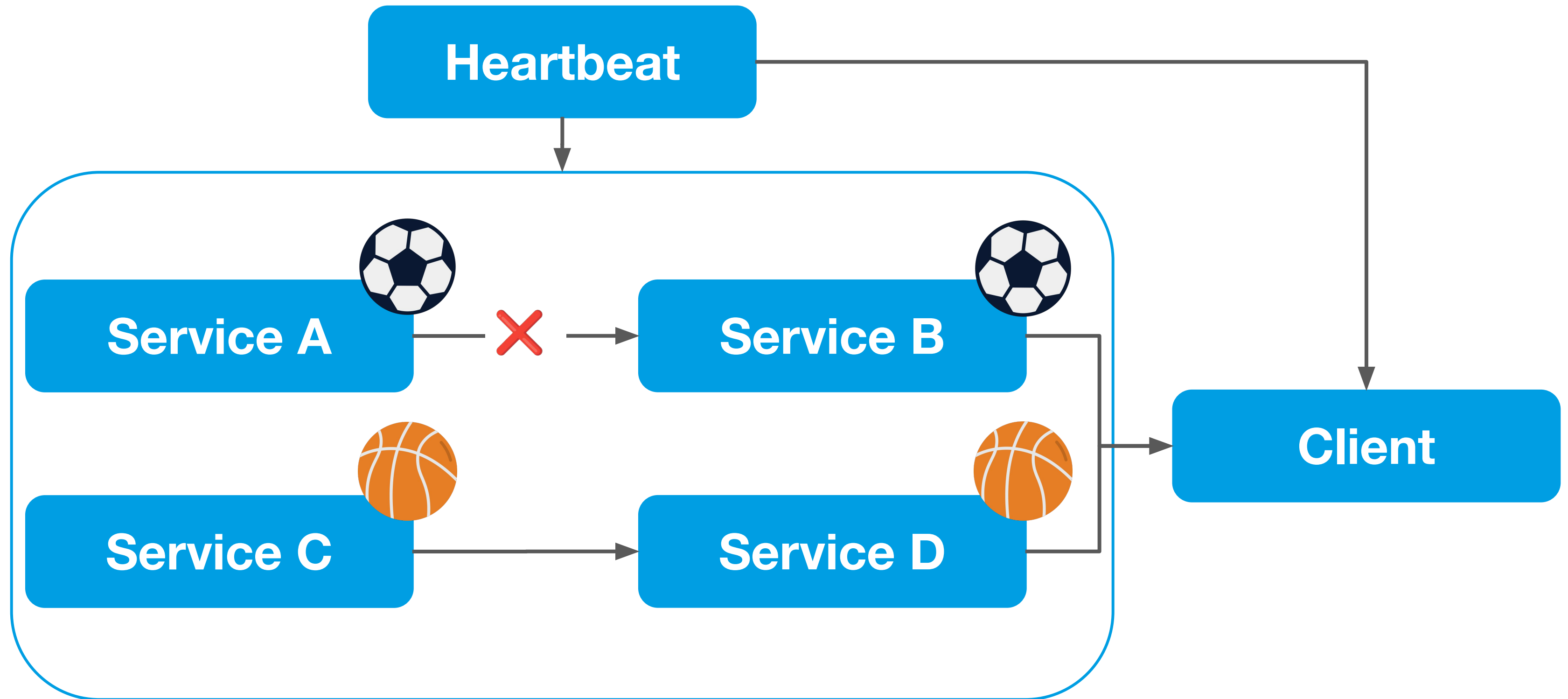
Как было у нас



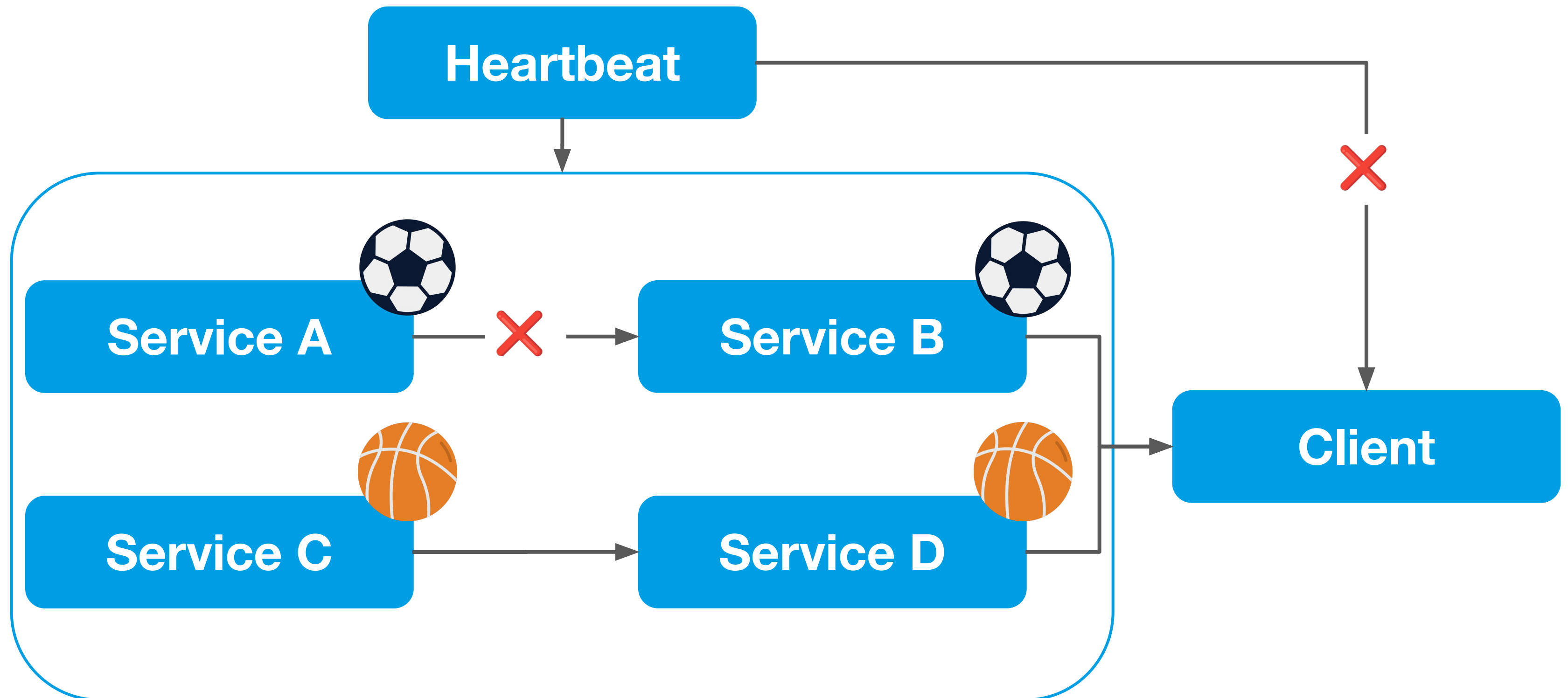
Как было у нас



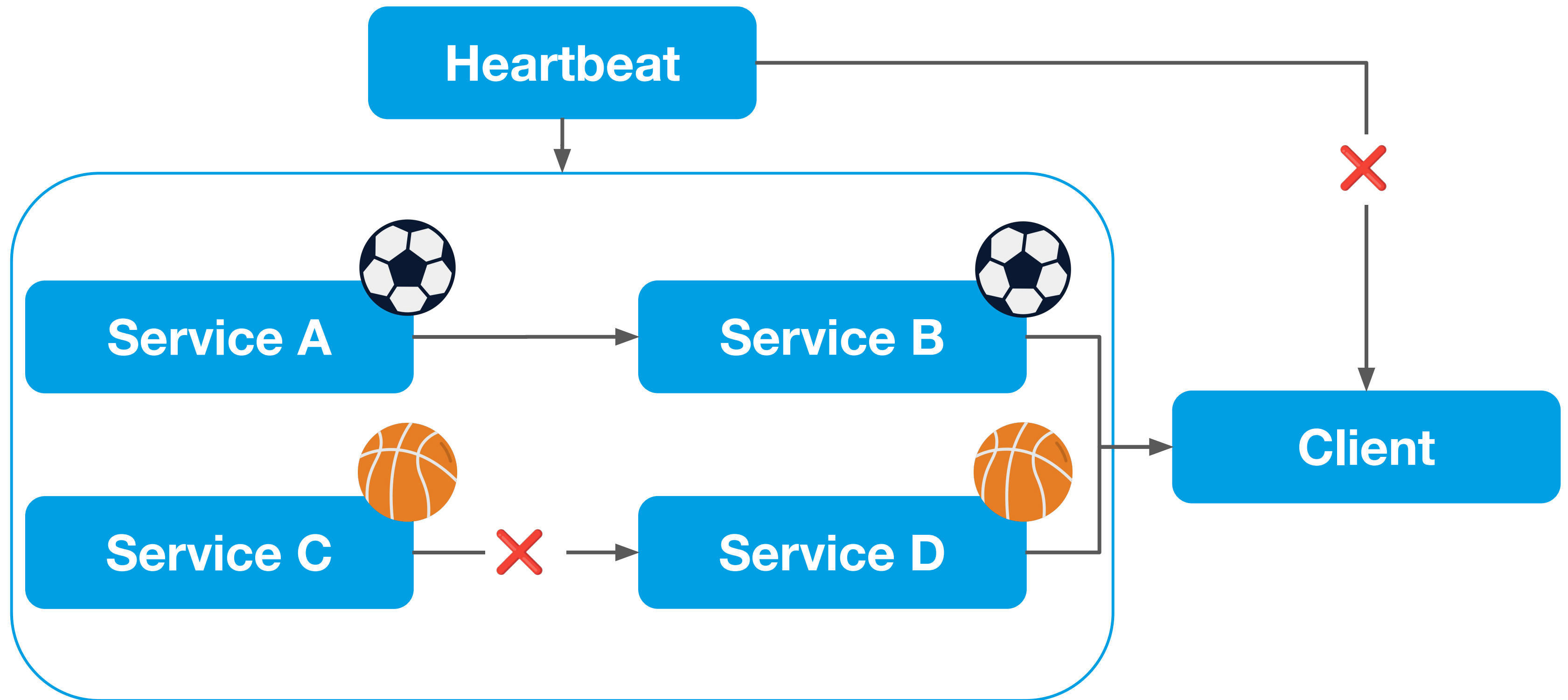
Как было у нас



Как было у нас



Как было у нас



Проблема такого heartbeat'a

- Капризный при росте сложности системы
- Неявное связывание независимых сервисов
- Система теряет доступность там, где не должна

Проблема такого heartbeat'a

Представьте, как такой heartbeat наблюдал бы за пациентом у которого обнаружилась температура 36.8?



Задача

Переделать heartbeat так, чтобы:

- Разделить влияние сервисов на систему

Задача

Переделать heartbeat так, чтобы:

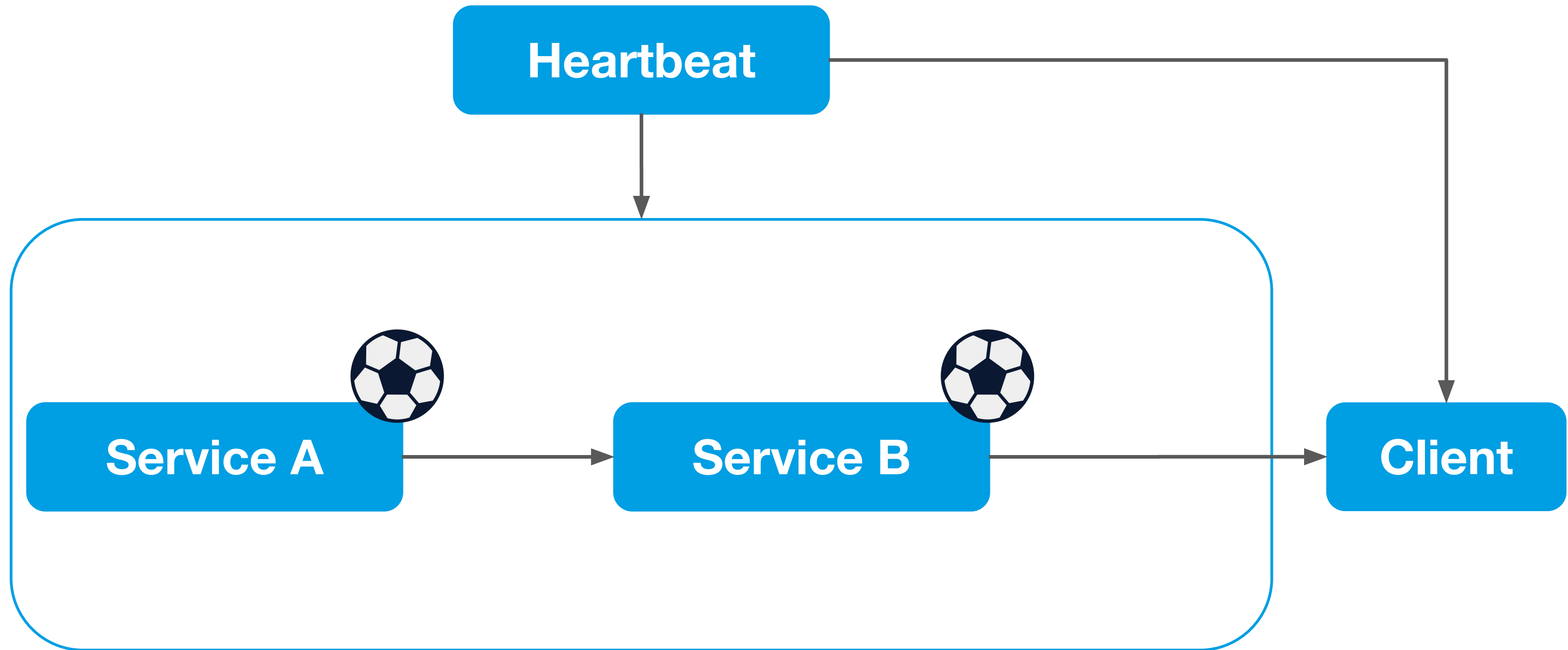
- Разделить влияние сервисов на систему
- Сделать подсистему расширяемой и настраиваемой

Задача

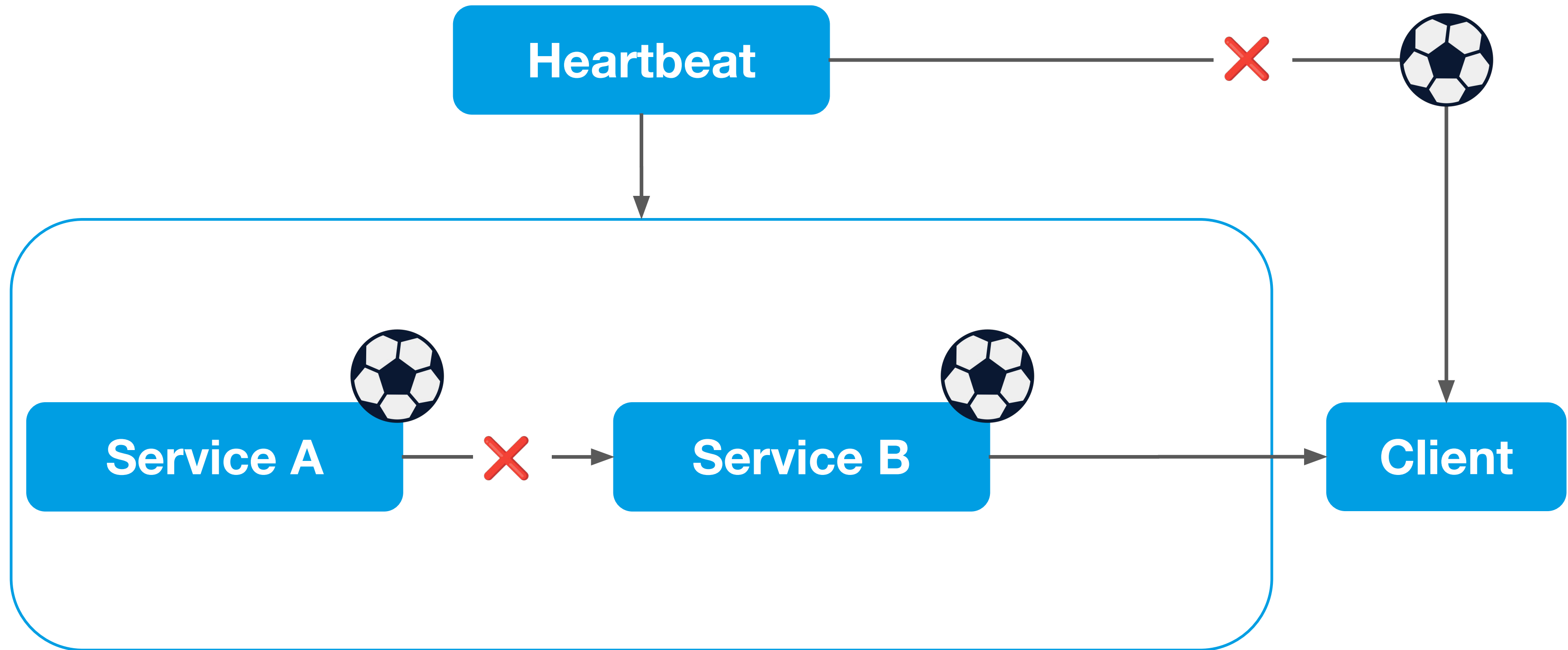
Переделать heartbeat так, чтобы:

- Разделить влияние сервисов на систему
- Сделать подсистему расширяемой и настраиваемой
- Сохранить гарантии надежности

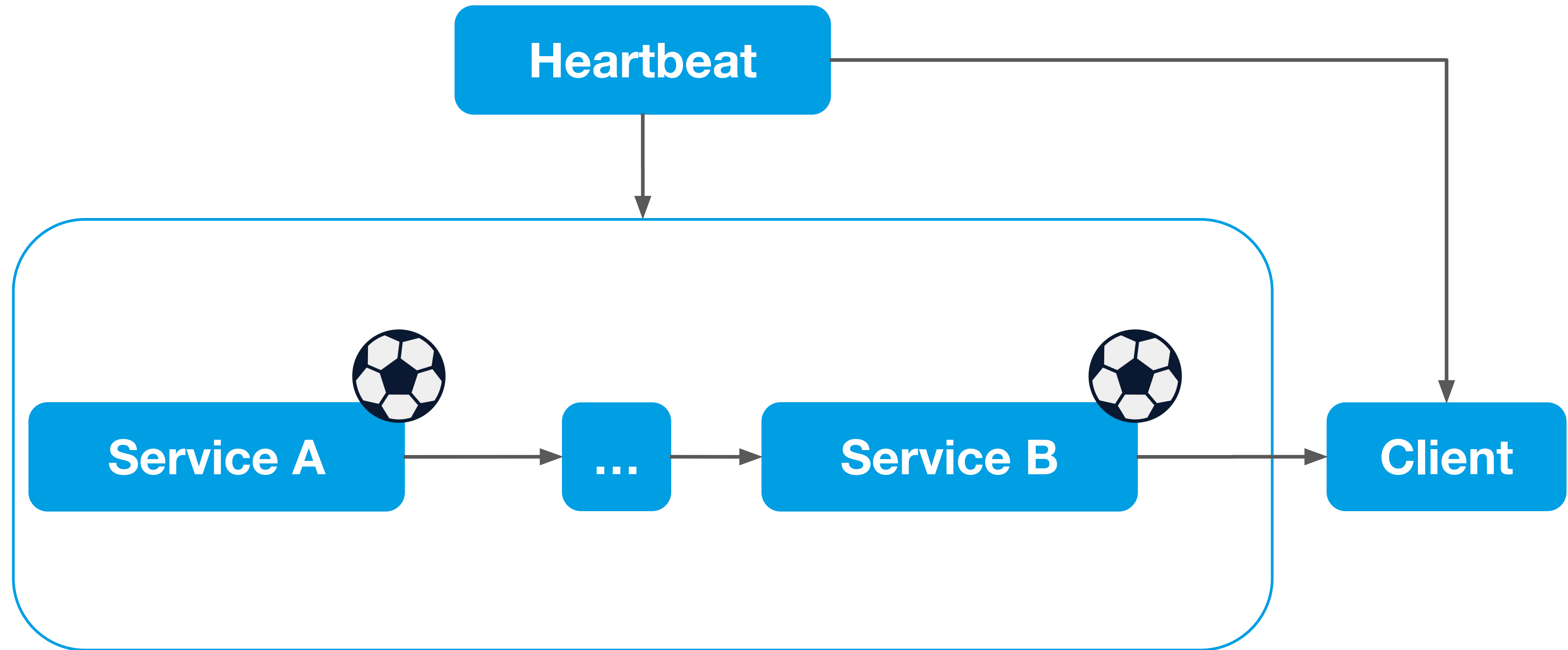
Гарантии надежности



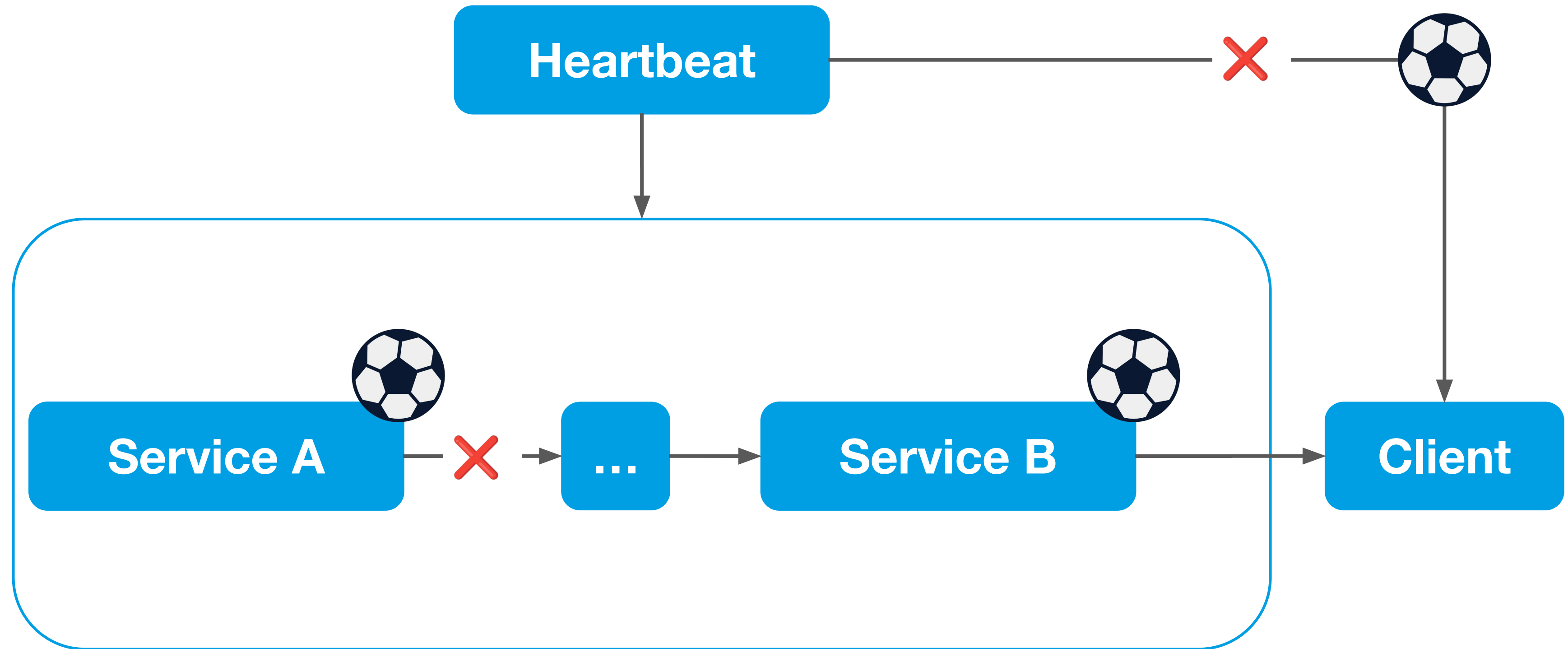
Гарантии надежности



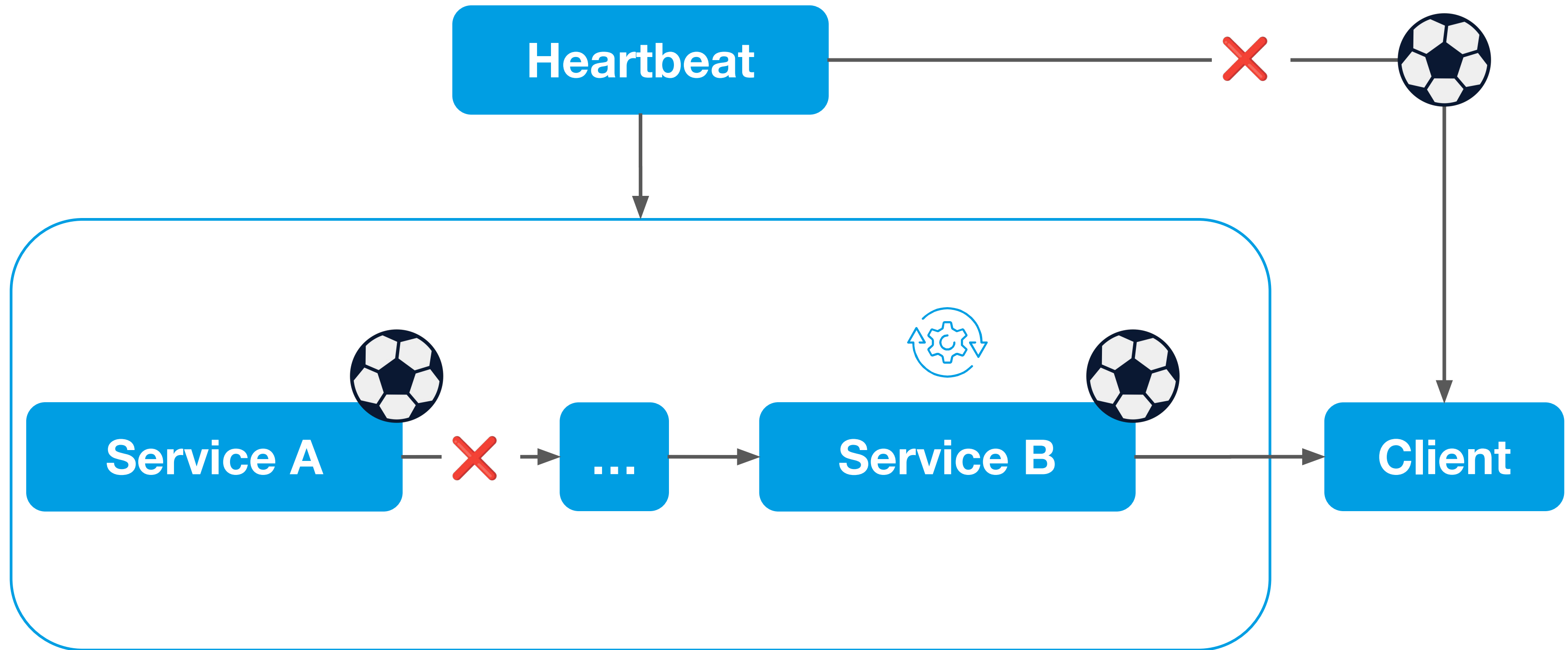
Гарантии надежности



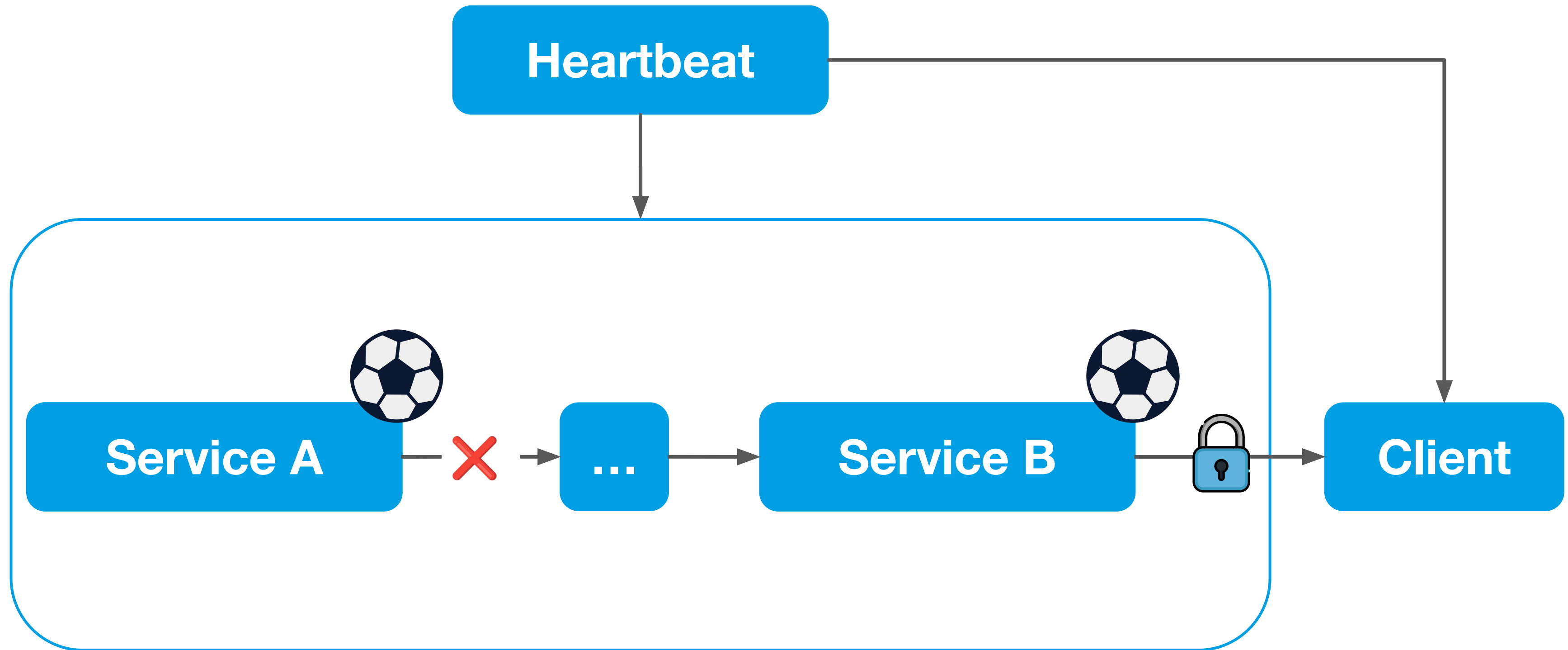
Гарантии надежности



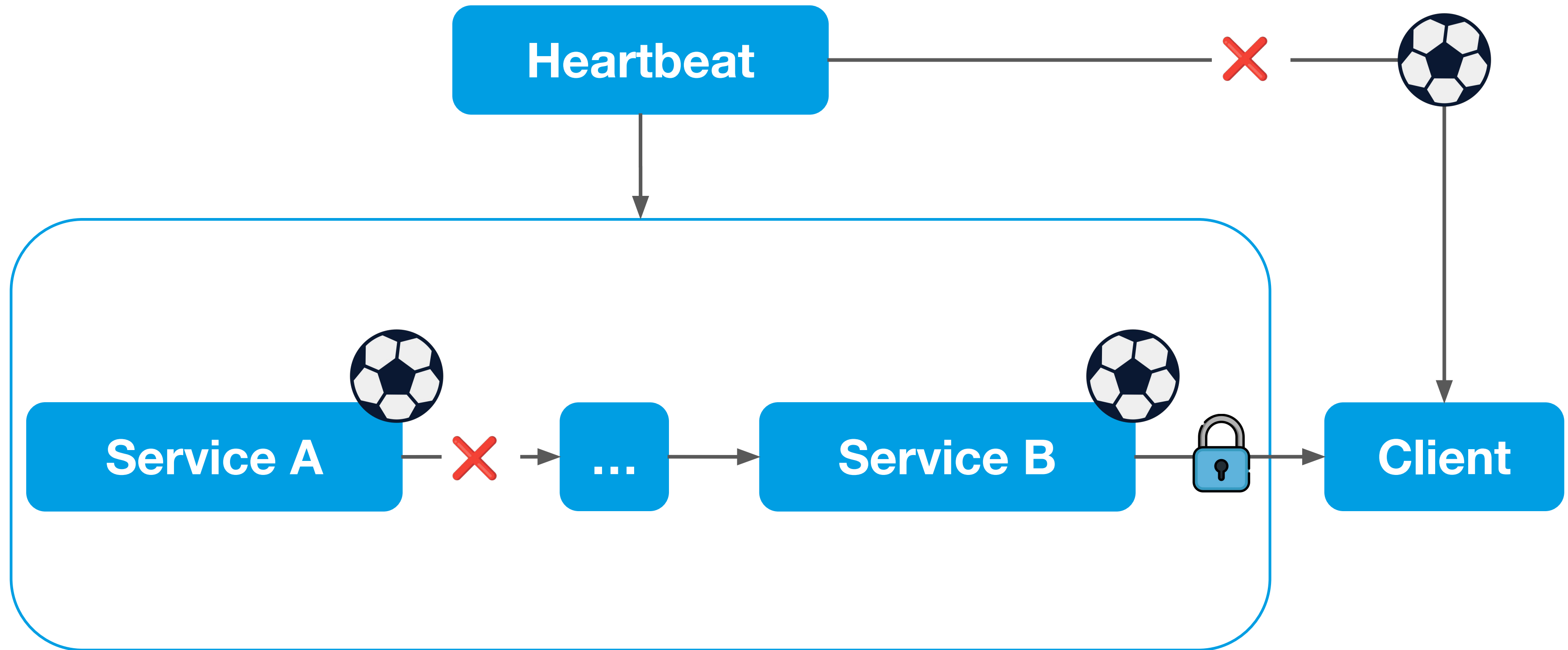
Гарантии надежности



Гарантии надежности



Гарантии надежности



Heartbeat решения

Фича / Решение	Status page (Instatus)	Monitoring (Uptime Kuma)	Service mesh (Istio)	Наше решение
Автореакция	✗	✗	✓	✓
Гранулярность	✗	✗	✗	✓
Чистота данных	✗	✗	✗	✓

Heartbeat решения

Фича / Решение	Status page (Instatus)	Monitoring (Uptime Kuma)	Service mesh (Istio)	Наше решение
Автореакция	✗	✗	✓	✓
Гранулярность	✗	✗	✗	✓
Чистота данных	✗	✗	✗	✓

Heartbeat решения

Фича / Решение	Status page (Instatus)	Monitoring (Uptime Kuma)	Service mesh (Istio)	Наше решение
Автореакция	✗	✗	✓	✓
Гранулярность	✗	✗	✗	✓
Чистота данных	✗	✗	✗	✓

Heartbeat решения

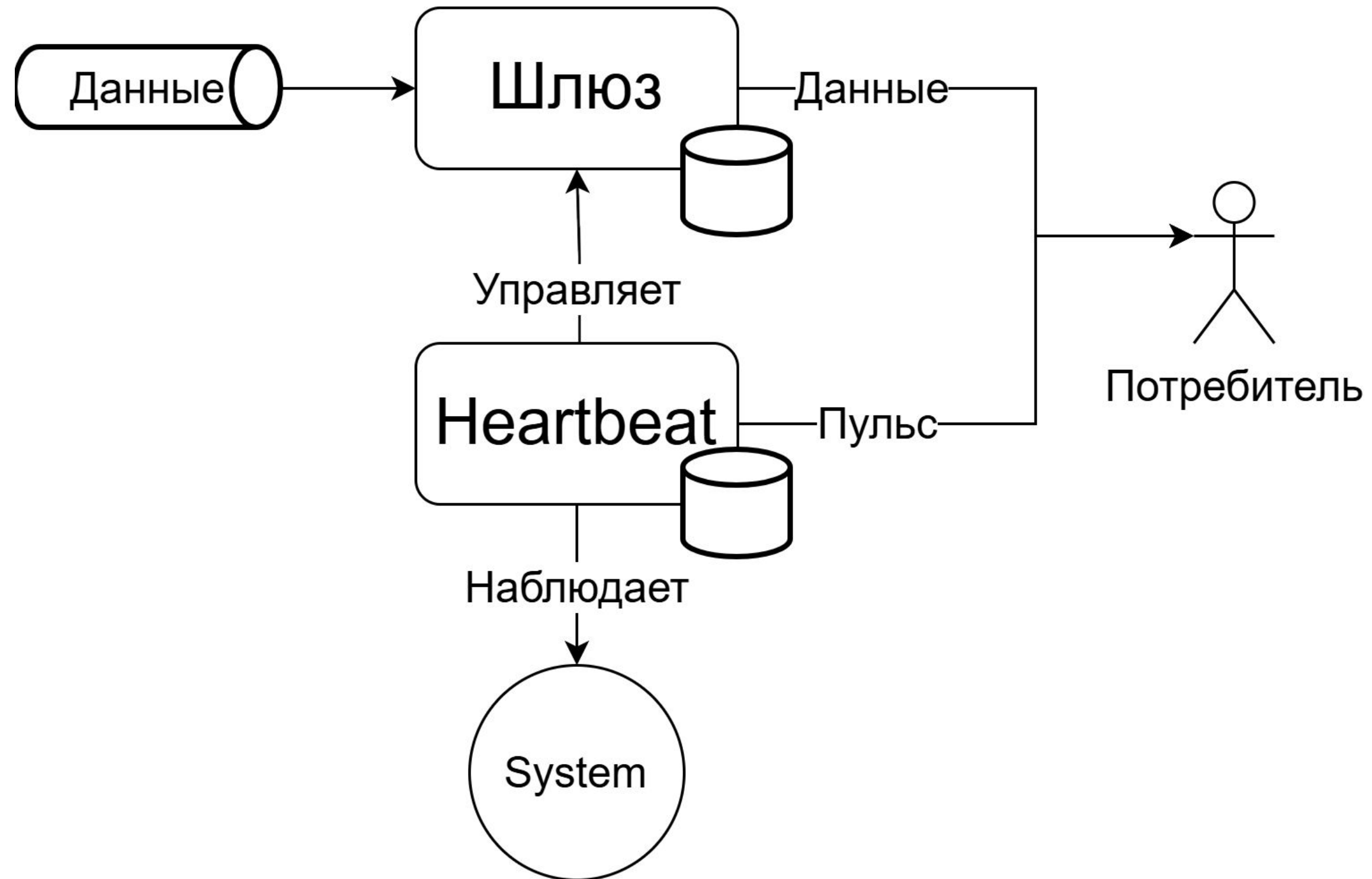
Фича / Решение	Status page (Instatus)	Monitoring (Uptime Kuma)	Service mesh (Istio)	Наше решение
Автореакция	✗	✗	✓	✓
Гранулярность	✗	✗	✗	✓
Чистота данных	✗	✗	✗	✓

Heartbeat управляет системой

- Не только пинг и мониторинг
- Полноценный компонент управления системой

Цель – продолжать приносить ценность как продукт

Heartbeat управляет системой



План действий

- Поделим сервисы на группы, поняв кто и как влияет на спорты и матчи (относительно просто, но есть нюансы)

План действий

- Поделим сервисы на группы, поняв кто и как влияет на спорты и матчи (относительно просто, но есть нюансы)
- Определим критерий неисправности (достаточно сложно)

План действий

- Поделим сервисы на группы, поняв кто и как влияет на спорты и матчи (относительно просто, но есть нюансы)
- Определим критерий неисправности (достаточно сложно)
- Определим не/приемлемое время простоя (совместно с бизнесом) (относительно просто)

План действий

- Поделим сервисы на группы, поняв кто и как влияет на спорты и матчи (относительно просто, но есть нюансы)
- Определим критерий неисправности (достаточно сложно)
- Определим не/приемлемое время простоя (совместно с бизнесом) (относительно просто)
- Определим критерий восстановления (достаточно сложно)

Поделим сервисы на группы

Придумаем единый формат описания влияния конкретного сервиса на матчи – **heartbeat спецификацию**

Heartbeat спецификация – это шаблон признаков по которому необходимо принять решение о блокировке матча

Поделим сервисы на группы

```
{  
  "hb_loss_time": "00:05:00",  
  "pattern_affected_matches": {  
    "sport_ids": [ 2 ],  
    "match_statuses": [ "active" ],  
    "required_sources": [ "some_source" ],  
    "..."  
  }  
}
```

Поделим сервисы на группы

```
{  
  "hb_loss_time": "00:05:00",  
  "pattern_affected_matches": {  
    "sport_ids": [ 2 ],  
    "match_statuses": [ "active" ],  
    "required_sources": [ "some_source" ],  
    "..."  
  }  
}
```

Поделим сервисы на группы

```
{  
  "hb_loss_time": "00:05:00",  
  "pattern_affected_matches": {  
    "sport_ids": [ 2 ],  
    "match_statuses": [ "active" ],  
    "required_sources": [ "some_source" ],  
    "..."  
  }  
}
```


Поделим сервисы на группы

```
{  
  "hb_loss_time": "00:05:00",  
  "pattern_affected_matches": {  
    "sport_ids": [ 2 ],  
    "match_statuses": [ "active" ],  
    "required_sources": [ "some_source" ],  
    "..."  
  }  
}
```

Поделим сервисы на группы

```
{  
  "hb_loss_time": "00:05:00",  
  "pattern_affected_matches": {  
    "sport_ids": [ 2 ],  
    "match_statuses": [ "active" ],  
    "required_sources": [ "some_source" ],  
    "..."  
  }  
}
```

Как понять, что сервису плохо?



Health check

- Microsoft.Extensions.Diagnostics.HealthChecks (nuget)

```
var builder = WebApplication.CreateBuilder(args);  
builder.Services.AddHealthChecks();
```

```
var app = builder.Build();  
app.MapHealthChecks("/hc");  
app.Run();
```

Health check

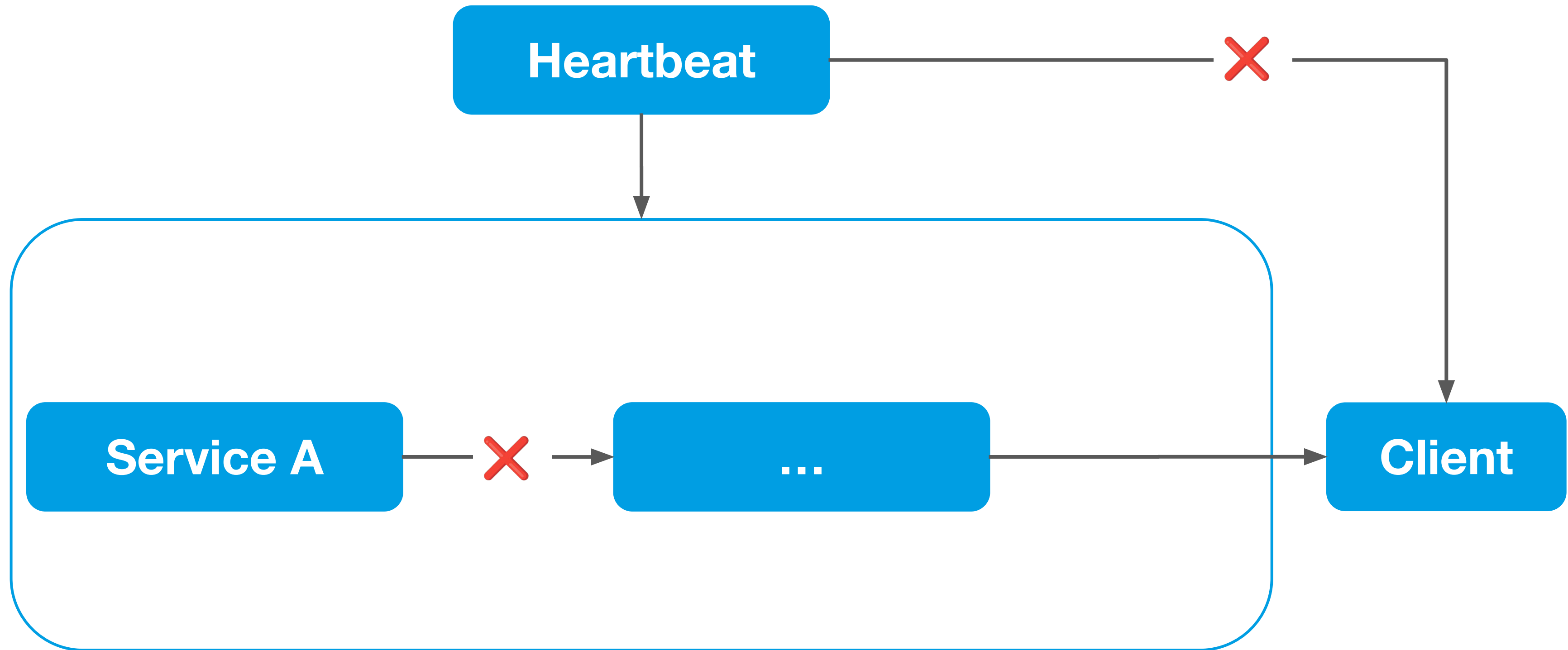
- ASP.NET Core.HealthChecks.Npgsql (nuget) и др.

```
builder.Services.AddHealthChecks()  
    .AddCheck<MyCheck>(..., tags: ["tag1"]) // IHealthCheck  
    .AddNpgsql(  
        builder.Configuration.GetConnectionString("DB"),  
        tags: ["tag2"]);
```

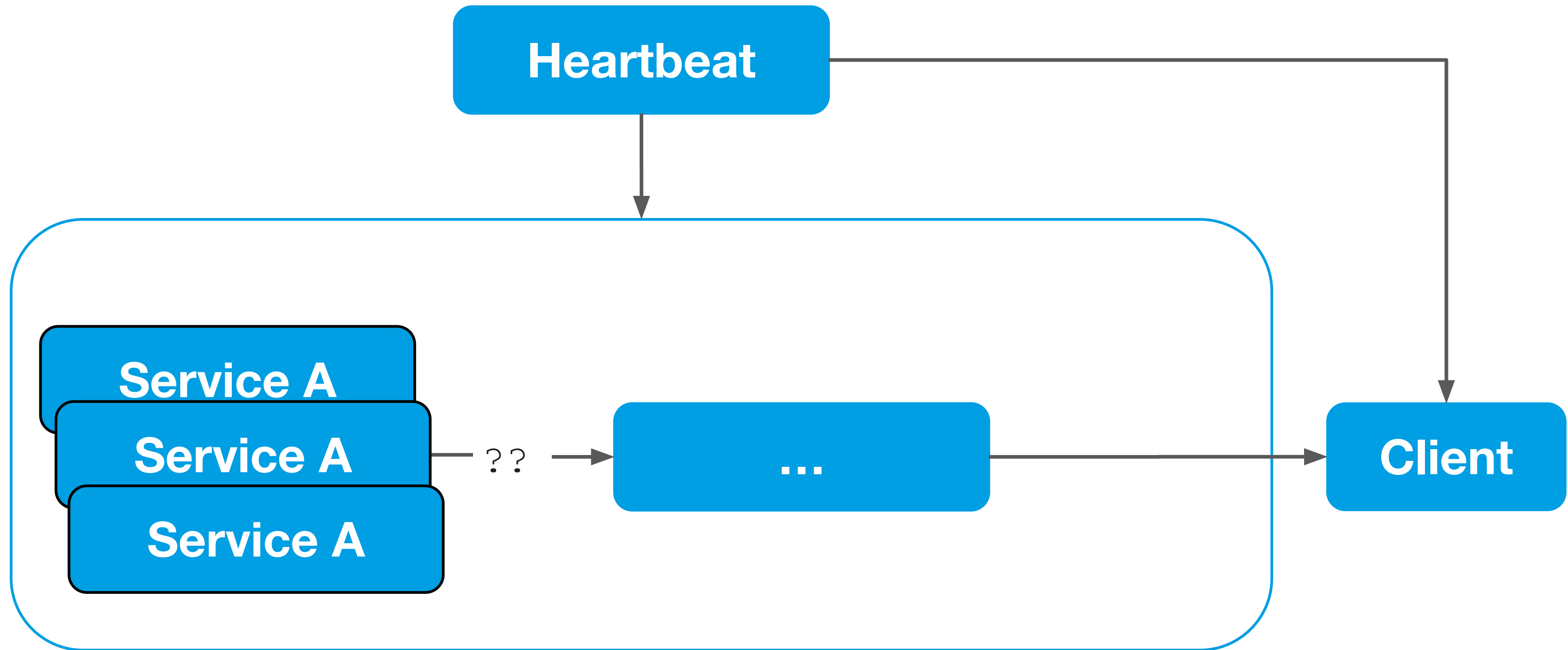

Health check

```
app.MapHealthChecks("/hc/live", new()  
{  
    Predicate = (opt) => opt.Tags.Contains("tag1")  
});  
app.MapHealthChecks("/hc/ready", new()  
{  
    Predicate = (opt) => opt.Tags.Contains("tag2")  
});
```

Health check



Health check



Kubernetes!



Kubernetes probes

spec:

containers:

- name: my-dotnet-service

startupProbe:

httpGet:

path: /hc

port: 8080

failureThreshold: 3 # Даем 3 попытки

periodSeconds: 10 # Раз в 10 секунд

Kubernetes probes

spec:

containers:

- name: my-dotnet-service

livenessProbe: # Если не прошли — рестарт контейнера

httpGet:

path: /hc # Или hc/live

port: 8080

initialDelaySeconds: 15 # Можно ждать здесь без startup

periodSeconds: 10

Kubernetes probes

spec:

containers:

- name: my-dotnet-service

readinessProbe: # Влияет на передачу трафика

httpGet:

path: /hc # Или hc/ready

port: 8080

periodSeconds: 5

Kubernetes deployment status

kind: Deployment

metadata:

name: my-dotnet-service

labels:

is-observable: 'true'

spec:

replicas: 2

status:

replicas: 2 updatedReplicas: 2

readyReplicas: 2 availableReplicas: 2

Kubernetes deployment status

kind: Deployment

metadata:

name: my-dotnet-service

labels:

is-observable: 'true'

spec:

replicas: 2

status:

replicas: 2 updatedReplicas: 2

readyReplicas: 2 availableReplicas: 2

Kubernetes deployment status

kind: Deployment

metadata:

name: my-dotnet-service

labels:

is-observable: 'true'

spec:

replicas: 2

status:

replicas: 2 updatedReplicas: 2

readyReplicas: 2 availableReplicas: 2

Kubernetes deployment status

kind: Deployment

metadata:

name: my-dotnet-service

labels:

is-observable: 'true'

spec:

replicas: 2

status:

replicas: 2 updatedReplicas: 2

readyReplicas: 2 availableReplicas: 2

Kubernetes deployment status

kind: Deployment

metadata:

name: my-dotnet-service

labels:

is-observable: 'true'

spec:

replicas: 2

status:

replicas: 2

updatedReplicas: 2

readyReplicas: 2

availableReplicas: 2

Kubernetes client

- KubernetesClient (nuget)

```
var deployments = await
    _kube.AppsV1.ListNamespacedDeploymentAsync(
        _configuration.TargetNamespace,
        labelSelector: _configuration.TargetLabelSelector, ...);
```

Kubernetes client

```
return deployments.Items.Select(x =>
    new ServiceDescription(
        new(x.Name()),
        new(x.Status.AvailableReplicas ?? 0,
            x.Status.UpdatedReplicas ?? 0,
            expectedReplicas: x.Spec.Replicas ?? 0,
            totalReplicas: x.Status.Replicas ?? 0),
        x.Spec.Selector.MatchLabels.AsReadOnly()));
```

Kubernetes client

```
if (description.ReplicasParameters.AvailableReplicas ==  
    description.ReplicasParameters.ExpectedReplicas)  
{  
    // ToDo  
}
```

PROFIT ???



Подводные камни

- Обновления
 - Важно сверяться именно со `spec.replicas`, а не `status.replicas`
 - StatefulSet не поддерживает RollingUpdate
 - Сверять `status.updated_replicas`

Подводные камни

- Обновления
- Рескалирование (особенно актуально вместе с KEDA)
 - При повышении/понижении реплик могут быть проблемы
 - Не получается отследить через `updated_replicas` и `observerd_generation`
 - Придется запоминать предыдущее состояние для отслеживания

Подводные камни

- Обновления
- Рескалирование (особенно актуально вместе с KEDA)
- Инфраструктурное обслуживание (maintenance)
 - Pod Disruption Budget (PDB)
 - High availability – 2+ реплики
 - Пришлось смотреть в состояние конкретных pod'ов

Критерий восстановления

- Не всегда работающий сервис отвечает требованиям (лаг и т.д.)
- Мы решили прийти к понятию время восстановления

Задачи

- Разделить влияние сервисов на систему
- Сделать подсистему расширяемой и настраиваемой
- Сохранить гарантии надежности

Задачи

- ✓ Разделить влияние сервисов на систему
 - Сделать подсистему расширяемой и настраиваемой
 - Сохранить гарантии надежности

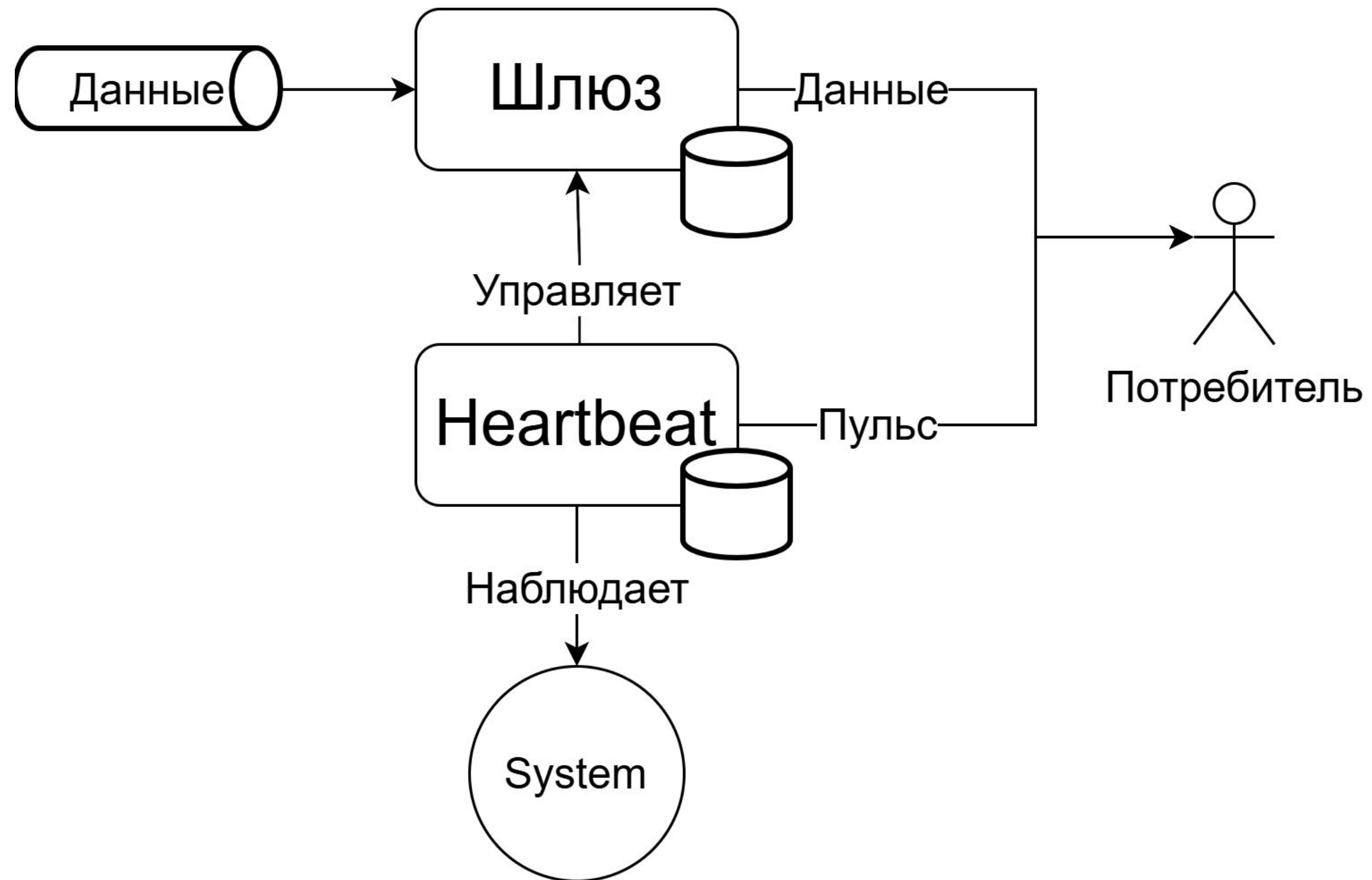
Задачи

- ✓ Разделить влияние сервисов на систему
- ✓ Сделать подсистему расширяемой и настраиваемой
 - Сохранить гарантии надежности

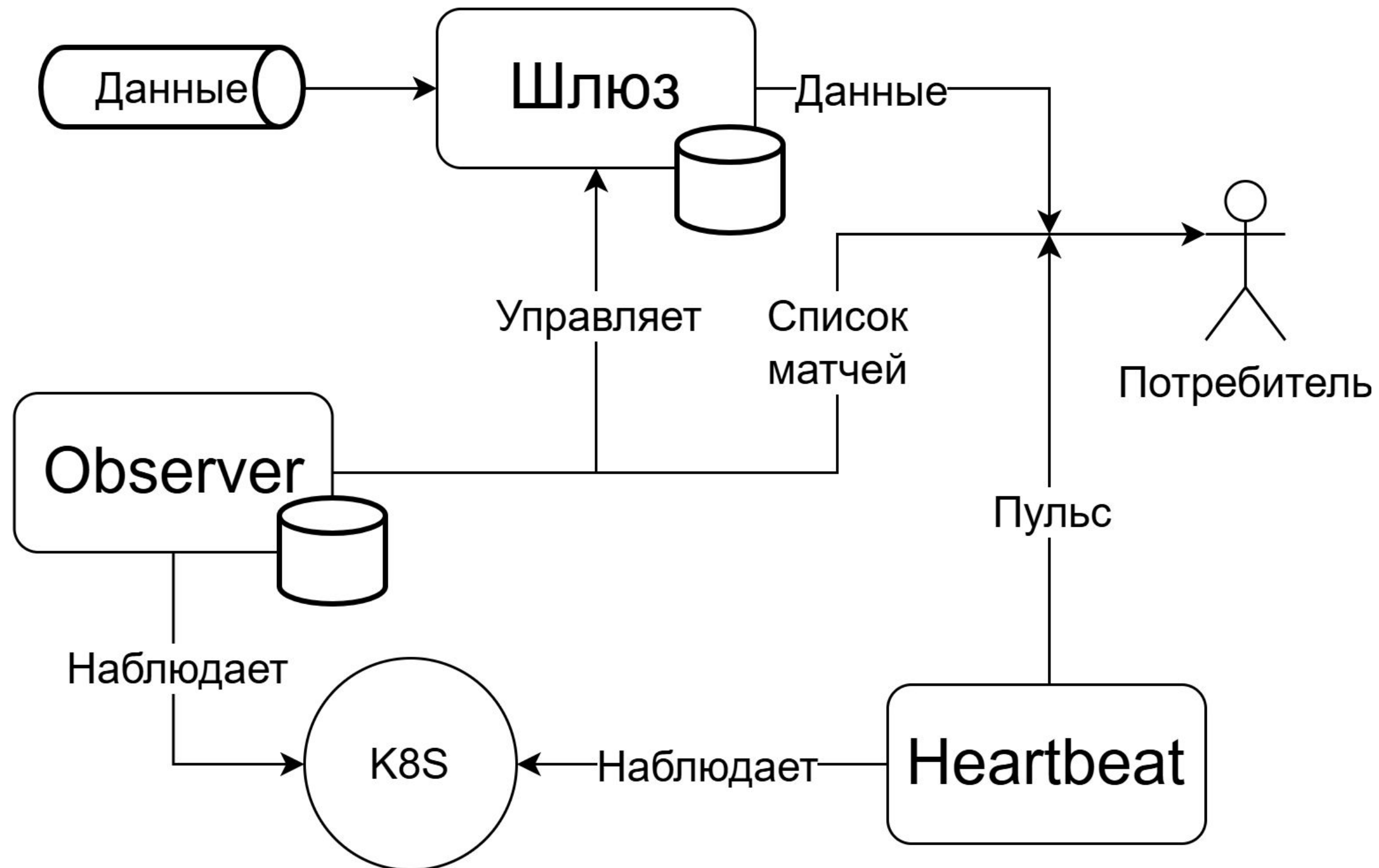
Задачи

- ✓ Разделить влияние сервисов на систему
- ✓ Сделать подсистему расширяемой и настраиваемой
- ✓ Сохранить гарантии надежности

Критические сервисы



Критические сервисы



По итогу

- Система которая гранулярно сообщает о неисправностях и блокирует трафик
- Уменьшает время простоя системы
- Уменьшает кол-во ручных действий со стороны сотрудников мониторинга/поддержки

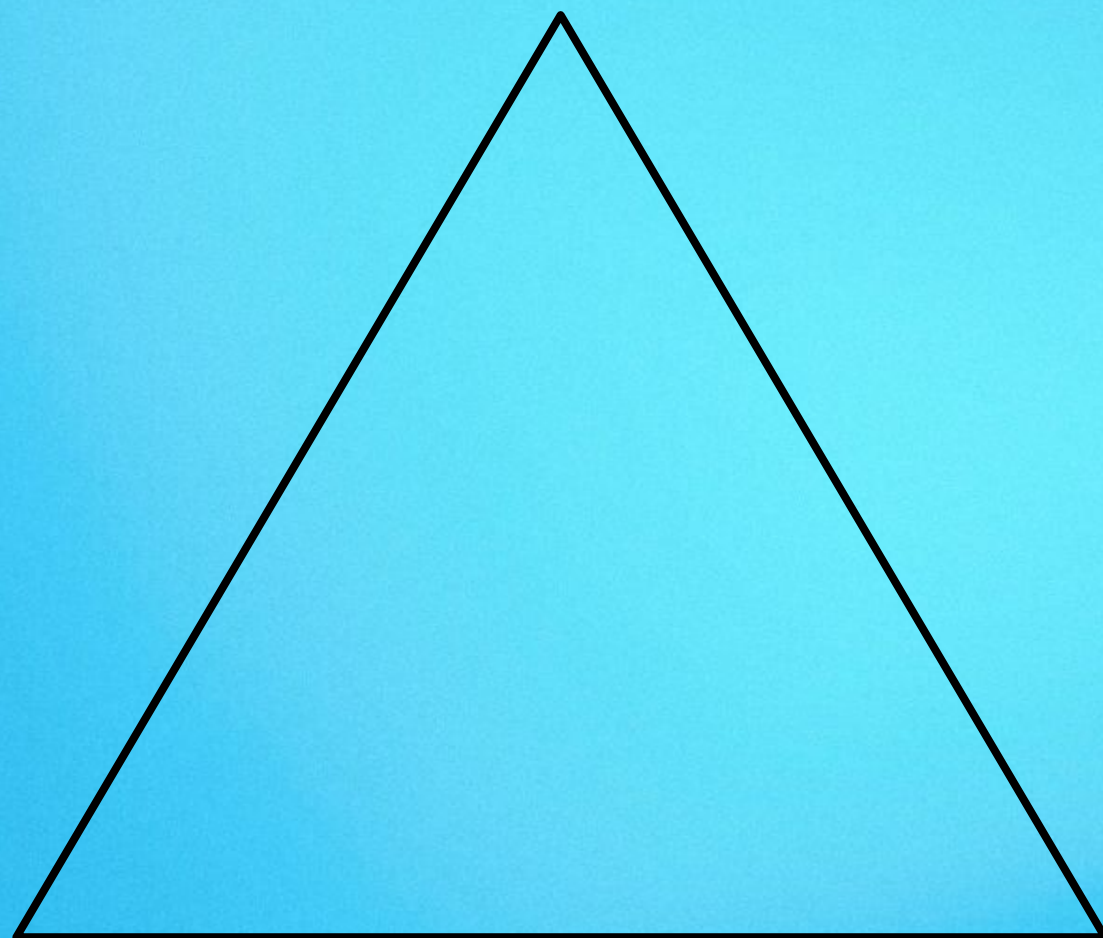
Что можно сделать ещё?

- Разделять отказ и деградацию
- Минимизация восстановления / уход от таймеров
- Не усложнять – пингуйте, если достаточно

Польза

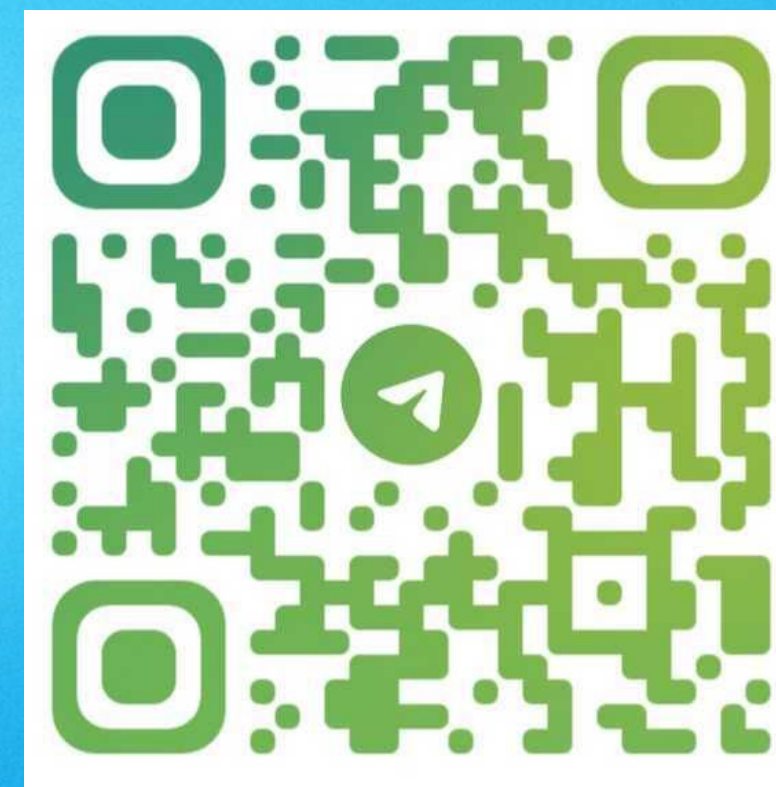
- Heartbeat – это страховка
- Поможет оперативно среагировать и предоставить агрегированную информацию
- Локализует проблему
- Теперь звонки длятся меньше или отсутствуют

Скорость
реагирования



Гибкость

Надежность



 **@centuriin**